DSA03-NATURAL LANGUAGE PROCESSING

ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

NAME:K.BALAJI

REG.NO:192324180

Submission Date: 05/11/2026

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis

1. Aim

To analyse the linguistic challenges involved in searching hospital records and identify suitable NLP models, algorithms and processing techniques for text normalisation, entity recognition and morphological analysis.

2. Levels of Linguistic Analysis

Natural Language Processing consists of six major levels of linguistic analysis.

Level	Description	Hospital Example	Requirement
Phonology	Studies speech sounds and pronunciation.	Different pronunciations of medical terms.	Not required initially
Morphology	Studies word structure, roots and affixes.	Fracture, fractures, fractured.	Required
Syntax	Studies sentence structure and grammar.	Patient has severe fever.	Partially required
Semantics	Studies the meaning of words and sentences.	Identifying the meaning of renal disease.	Partially required
Pragmatics	Studies meaning based on context and intention.	Understanding a doctor's intention from a note.	Not required initially
Discourse	Studies relationships between multiple sentences.	Connecting patient history with discharge notes.	Not required initially

Explanation:

1. Phonology: It deals with speech sounds. Since the initial hospital records are text-based, phonological analysis is not necessary.
2. Morphology: It identifies the root words and their variations. For example, fracture, fractures and fractured should be connected during searching.
3. Syntax: It identifies grammatical relationships between words. It can help understand the structure of clinical sentences.
4. Semantics: It deals with the meaning of words. For example, nephrectomy is a surgical removal of a kidney, while renal refers to the kidney.
5. Pragmatics: It understands the intended meaning using context. It is not essential for the initial text normalisation layer.
6. Discourse: It connects information across multiple sentences and documents. It can be added in future development.

Conclusion: Morphology and basic syntactic analysis are important for this project. Structured entity recognition and limited semantic handling are also required.

3. Three Model Families Used in NLP

Three important model families are state machines, formal rule systems and probabilistic models.

Model Family	Description	Hospital Application	Why Other Models Are Less Suitable
State Machines	Recognise patterns using states and transitions.	Validate patient IDs, dates and medical codes.	Probabilistic models are unnecessary for exact format validation.
Formal Rule Systems	Apply predefined linguistic rules.	Identify word stems, prefixes and suffixes.	State machines alone do not conveniently represent all morphological rules; probabilistic models may produce uncertain analyses.
Probabilistic Models	Use probability to predict the most likely result.	Identify misspelt medical terms and ambiguous abbreviations.	Exact rules are more reliable for fixed formats; manual rules are difficult to maintain for highly variable noisy language.

Example:

- State Machine: Validate patient ID PAT12345.
- Formal Rule System: Convert fractures into fracture + Noun + Plural.
- Probabilistic Model: Identify the most likely intended word when OCR produces an incorrect spelling.

4. Ambiguity in Natural Language Processing

Ambiguity occurs when a word, sentence or text has more than one possible interpretation.

The four major ambiguities in hospital records are:

Ambiguity	Explanation	Hospital Example
Lexical Ambiguity	A word has multiple meanings.	Discharge can mean hospital release or fluid discharge.
Morphological Ambiguity	A word can have multiple morphological interpretations.	Found can be a past-tense verb or an adjective.
Segmentation Ambiguity	Text can be divided into tokens in different ways.	500mg can be separated into 500 and mg.

Abbreviation Ambiguity	An abbreviation can have multiple expansions.	BP can mean Blood Pressure or another term depending on context.

Why is ambiguity a central problem?

Natural language is complex because the same word may have different meanings depending on its context. Therefore, the system must identify the correct interpretation before normalising and retrieving hospital records.

5. Search in NLP

Search plays an important role in regular expressions, automata and text matching.

A. State-Space Search in Automata

- An automaton moves from one state to another based on the input symbol.
- It checks whether the input string follows the required pattern.
- Example: Validating a patient ID using DFA.

B. Backtracking in Pattern Matching

- Backtracking tries alternative matching possibilities when a previous match fails.
- It is used by some regular expression engines.
- Example: Matching different dosage formats in a clinical sentence.

Resources required for NLP search:

Resource	**Requirement**
Time	Processing time required for matching and searching.
Memory	Storage required for automaton states, lexicons and tokens.
Annotated Data	Labelled word forms and clinician queries for evaluation.
Expert Effort	Time required to develop patterns, linguistic rules and medical vocabulary.

6. Proposed NLP Pipeline – Labelled Diagram

The following diagram represents the proposed hospital text normalisation and search system.

INPUT

Hospital Clinical Records

Discharge summaries, prescriptions, operation notes and nursing observations

Stage 1: Regular Expression Recognition

Recognises patient IDs, dates, drug strengths and laboratory values.

Regex

Stage 2: Tokenisation

Splits clinical text into individual tokens.

Regular Expressions

Stage 3: Text Normalisation

Converts text into a standard format and expands abbreviations.

Rule-Based System

Stage 4: Entity Validation

Validates structured clinical information.

Finite State Automata (DFA/NFA)

Stage 5: Morphological Analysis

Identifies stems, prefixes, suffixes and grammatical features.

Finite-State Transducer (FST)

Stage 6: Stemming

Reduces related word forms to searchable stems.

Porter Stemmer

OUTPUT

Normalised and Searchable Hospital Records

Improved clinical information retrieval

TASK 2 – Basic Regular Expression Patterns

1. Aim

To study basic regular expression operators and develop patterns for identifying structured information in hospital records, such as patient IDs, dates, drug names, dosage frequencies, laboratory values and other clinical entities.

2. Basic Regular Expression Operators

Regular Expressions (Regex) are patterns used to identify, search and validate specific sequences of characters in text.

Operator	**Meaning**	**Clinical Example**
Concatenation	Combines characters in sequence.	PAT matches PAT.

Disjunction |	Matches either of two alternatives.	fever|cough
Kleene Star *	Matches zero or more repetitions.	A*
Plus +	Matches one or more repetitions.	[0-9]+
Optionality ?	Matches zero or one occurrence.	colou?r
Character Class []	Matches one character from a group.	[ABC]
Range -	Matches characters within a range.	[0-9]
Negated Class [^]	Matches characters not in the specified group.	[^0-9]+
Anchors ^ and $	Match the beginning and end of a string.	^PAT[0-9]+$
Counters {n,m}	Specifies repetition limits.	[0-9]{2,4}

Explanation with examples

1. Concatenation: PAT[0-9] matches PAT followed by one digit.
2. Disjunction: fever|cough matches either fever or cough.
3. Kleene Star: AB* matches A, AB, ABB and so on.
4. Plus: [0-9]+ matches one or more digits.
5. Optionality: colou?r matches color and colour.
6. Character Class: [ABC] matches A, B or C.
7. Range: [0-9] matches any digit from 0 to 9.
8. Negated Class: [^0-9]+ matches one or more non-digit characters.
9. Anchors: ^PAT[0-9]+$ validates a complete patient ID.
10. Counters: [0-9]{2,4} matches a number containing 2 to 4 digits.

3. Regular Expressions for 12 Hospital Entity Types

The following patterns are designed for the hospital record system.

3.1 Pattern Inventory

Each regular expression has two conditions:

- Must match: The valid clinical information accepted by the pattern.
- Must not match: Invalid or unrelated information that should be rejected.

The following 12 patterns cover the required hospital entities.

No.	Entity type	Regular expression	Example that must match
1	Patient ID	PAT[0-9]{5}	PAT12345
2	Admission / Discharge Date	(?:[0-9]{4}-[0-9]{2}-[0-9]{2}|[0-9]{2}/[0-9]{2}/[0-9]{4})	2026-11-06
3	Drug Name with Strength	(?:paracetamol|amoxicillin|ibuprofen)\s+[0-9]+(?:\.[0-9]+)?\s*(?:mg|mcg|g)	paracetamol 500mg
4	Dosage Frequency	(?:OD|BD|TDS|QID|BID|once daily|twice daily)	twice daily
5	Laboratory Value with Unit	[A-Za-z][A-Za-z0-9-]*\s*[:=]?\s*[0-9]+(?:\.[0-9]+)?\s*(?:mg/dL|g/dL|mmol/L|U/L)	Glucose: 120 mg/dL
6	Blood Pressure	[0-9]{2,3}/[0-9]{2,3}\s*mmHg	120/80 mmHg
7	Temperature	[0-9]{2}(?:\.[0-9])?\s*°?[CF]	37.5°C
8	ICD Code	[A-Z][0-9]{2}(?:\.[0-9A-Z]{1,4})?	E11.9
9	Ward and Bed	Ward\s+[A-Za-z0-9-]+\s*,?\s*Bed\s+[A-Za-z0-9-]+	Ward A, Bed 12
10	Contact Number	(?:\+91[-]?)?[6-9][0-9]{9}	9876543210
11	Patient Age	[0-9]{1,3}\s*(?:years?|yrs?)	45 years
12	Symptom Duration	[0-9]+\s*(?:hours?|days?|weeks?|months?)	3 days

Note: The drug-name pattern uses a small illustrative vocabulary. A real hospital system needs a larger validated drug lexicon. Date patterns recognise formats, but calendar validity must be checked separately.

3.2 What Each Pattern Must Not Match

No.	Entity	Must not match	Reason
1	Patient ID	PAT123	Only three digits
2	Date	2026-15-40	Invalid calendar date
3	Drug strength	paracetamol 500xyz	Invalid unit
4	Dosage frequency	FIVE TIMES	Not in the defined frequency vocabulary
5	Laboratory value	Glucose: 120 abc	Invalid unit
6	Blood pressure	120/abc mmHg	Diastolic value must be numeric
7	Temperature	105XYZ	Invalid temperature unit
8	ICD code	123.45	Must begin with a letter
9	Ward and bed	Ward A Room 12	Missing Bed label
10	Contact number	123456	Insufficient digits
11	Patient age	25 days	Age pattern accepts years

12	Symptom duration	three days	Numeric value required

4. Greedy and Non-Greedy Matching

Greedy matching: A greedy quantifier tries to match the longest possible string.

Non-greedy matching: A non-greedy quantifier tries to match the shortest possible string that satisfies the pattern.

Consider this clinical sentence:

Patient prescribed paracetamol 500mg and ibuprofen 200mg after food.

Greedy pattern:

import retext = "Patient prescribed paracetamol 500mg and ibuprofen 200mg after food."greedy = re.findall(r".*mg", text)print(greedy)

Output:

['Patient prescribed paracetamol 500mg and ibuprofen 200mg']

Non-greedy pattern:

import retext = "Patient prescribed paracetamol 500mg and ibuprofen 200mg after food."non_greedy = re.findall(r".*?mg", text)print(non_greedy)

Output:

['Patient prescribed paracetamol 500mg',

' and ibuprofen 200mg']

Interpretation:

- .* consumes as much text as possible before reaching the final mg.
- .*? consumes as little text as possible before reaching each mg.

Therefore, non-greedy matching is more useful when extracting multiple dosage expressions from a clinical sentence. However, the extraction pattern should also be designed around the expected drug and dosage structure.

5. Operator Precedence and Grouping

Regular expression operators have precedence rules. Quantifiers bind to the preceding atom, concatenation binds more tightly than alternation, and grouping controls how alternatives are interpreted.

Example of an incorrect pattern:

fever|coughing

This pattern matches either fever or coughing.

It does not mean fever or cough followed by ing.

Correct grouped pattern:

(fever|cough)ing

This matches fevering or coughing.

Another example:

Incorrect:

PAT|DOC[0-9]+

This matches PAT alone or DOC followed by digits.

Correct:

(PAT|DOC)[0-9]+

This matches PAT12345 or DOC12345.

Conclusion: Parentheses are important because they control the scope of alternatives and prevent unexpected matching results.

6. Complete Python Implementation and Testing

The following Python program tests all 12 regular expression patterns using valid and invalid clinical examples.

```
import re

# 1. Define 12 clinical entity patterns
patterns = {
    "Patient ID": r"PAT[0-9]{5}",

    "Date": r"(?:[0-9]{4}-[0-9]{2}-[0-9]{2}|[0-9]{2}/[0-9]{2}/[0-9]{4})",

    "Drug Strength": r"(?:paracetamol|amoxicillin|ibuprofen)\s+[0-9]+(?:\.[0-9]+)?\s*(?:mg|mcg|g)",

    "Dosage Frequency": r"(?:OD|BD|TDS|QID|BID|once daily|twice daily)",
```

```
    "Lab Value": r"[A-Za-z][A-Za-z0-9-]*\s*[:=]?\s*[0-9]+(?:\.[0-
9]+)?\s*(?:mg/dL|g/dL|mmol/L|U/L)",

    "Blood Pressure": r"[0-9]{2,3}/[0-9]{2,3}\s*mmHg",

    "Temperature": r"[0-9]{2}(?:\.[0-9])?\s*°?[CF]",

    "ICD Code": r"[A-Z][0-9]{2}(?:\.[0-9A-Z]{1,4})?",

    "Ward and Bed": r"Ward\s+[A-Za-z0-9-]+\s*,?\s*Bed\s+[A-Za-z0-9-]+",

    "Contact Number": r"(?:\+91[- ]?)?[6-9][0-9]{9}",

    "Patient Age": r"[0-9]{1,3}\s*(?:years?|yrs?)",

    "Symptom Duration": r"[0-9]+\s*(?:hours?|days?|weeks?|months?)"
}

# 2. Test data: valid and invalid examples
test_data = {
    "Patient ID": [
        ("PAT12345", 1),
        ("PAT123", 0)
    ],

    "Date": [
        ("2026-11-06", 1),
        ("2026-15-40", 0)
    ],
```

```
"Drug Strength": [
    ("paracetamol 500mg", 1),
    ("paracetamol 500xyz", 0)
],

"Dosage Frequency": [
    ("twice daily", 1),
    ("FIVE TIMES", 0)
],

"Lab Value": [
    ("Glucose: 120 mg/dL", 1),
    ("Glucose: 120 abc", 0)
],

"Blood Pressure": [
    ("120/80 mmHg", 1),
    ("120/abc mmHg", 0)
],

"Temperature": [
    ("37.5°C", 1),
    ("105XYZ", 0)
],

"ICD Code": [
    ("E11.9", 1),
    ("123.45", 0)
```

```
    ],

    "Ward and Bed": [
        ("Ward A, Bed 12", 1),
        ("Ward A Room 12", 0)
    ],

    "Contact Number": [
        ("9876543210", 1),
        ("123456", 0)
    ],

    "Patient Age": [
        ("45 years", 1),
        ("25 days", 0)
    ],

    "Symptom Duration": [
        ("3 days", 1),
        ("three days", 0)
    ]
}

# 3. Evaluate each pattern
print("HOSPITAL REGEX TESTING RESULTS")
print("-" * 65)

for name, pattern in patterns.items():
```

```
tp = 0
fp = 0
fn = 0
tn = 0

for text, actual in test_data[name]:

    predicted = 1 if re.fullmatch(
        pattern, text, re.IGNORECASE
    ) else 0

    if predicted == 1 and actual == 1:
        tp += 1
    elif predicted == 1 and actual == 0:
        fp += 1
    elif predicted == 0 and actual == 1:
        fn += 1
    else:
        tn += 1

print("\nEntity:", name)
print("True Positives:", tp)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Negatives:", tn)

precision = tp / (tp + fp) if tp + fp else 0
recall = tp / (tp + fn) if tp + fn else 0
```

```
print("Precision:", round(precision, 2))
print("Recall:", round(recall, 2))
```

```
    HOSPITAL REGEX TESTING RESULTS
    ----------------------------------------
...
    Entity: Patient ID
    True Positives: 1
    False Positives: 0
    False Negatives: 0
    True Negatives: 1
    Precision: 1.0
    Recall: 1.0

    Entity: Date
    True Positives: 1
    False Positives: 1
    False Negatives: 0
    True Negatives: 0
    Precision: 0.5
    Recall: 1.0

    Entity: Drug Strength
    True Positives: 1
    False Positives: 0
    False Negatives: 0
    True Negatives: 1
    Precision: 1.0
    Recall: 1.0

    Entity: Dosage Frequency
    True Positives: 1
    False Positives: 0
    False Negatives: 0
    True Negatives: 1
    Precision: 1.0
    Recall: 1.0
```

6.1 Expected Output

The test dataset contains one valid and one invalid example for each entity.

Testing results

12

True positives

12

True negatives

0

False positives

0

False negatives

Illustrative test accuracy: 100%

This result applies only to the 24 constructed examples, not to the required 500-record clinical corpus.

7. Error Analysis of Regular Expression Patterns

When regular expressions are tested on real hospital records, three types of errors may occur.

Error type	**Meaning**	**Clinical example**	**Possible cause**
True Positive (TP)	Correctly identifies valid information.	PAT12345	Correct pattern
False Positive (FP)	Incorrectly accepts invalid information.	Invalid date 2026-15-40	Regex checks format but not calendar validity
False Negative (FN)	Fails to identify valid information.	Paracetamol 500 mg when the pattern expects a known spelling or supported format	Vocabulary or format variation

Specific linguistic causes of errors:

1. Spelling variations: paracetamol and incorrectly typed paracetmol.
2. Abbreviations: twice daily, BD and BID.
3. Formatting variations: 500mg and 500 mg.
4. OCR errors: PAT12345 may be recognised as PAT12S45.
5. Date variations: 06-11-2026 may not match the two supported date formats.

Note: The above are anticipated error causes. Their actual frequencies must be calculated using the hospital corpus.

TASK 3 – Regular Expressions for Tokenisation and Normalisation

1. Aim

To develop a regular-expression-based tokenizer and text normalisation system for hospital records. The system identifies meaningful tokens, handles clinical abbreviations, standardises dates and measurement units, and converts raw clinical text into a consistent searchable format.

2. Tokenisation Using Regular Expressions

Definition:

Tokenisation is the process of dividing a sentence or paragraph into smaller meaningful units called tokens.

For hospital records, tokens may include:

- Normal words: patient, fever, fracture.
- Hyphenated words: post-operative, follow-up.
- Abbreviations: Dr., BP, ECG.
- Decimal values: 37.5, 120.8.
- Drug strengths: 500mg, 250 mcg.
- Dates: 06/11/2026.

2.1 Tokenisation decisions

Token type	Example	Tokenisation decision
Normal words	Patient fever	Separate words
Hyphenated terms	Post-operative	Keep as one token
Abbreviations	Dr.	Keep as one token
Decimal values	37.5	Keep as one token
Units with numbers	500mg	Keep as one token
Dates	06/11/2026	Keep as one token
Punctuation	, . :	Separate or discard based on context

2.2 Single Regular Expression for Tokenisation

The following regular expression recognises the required clinical token types.

\d{2}/\d{2}/\d{4}|\d+(?:\.\d+)?(?:mg/dL|mmHg|mg|mcg|g|mL|°C|°F)\b|[A-Za-z]+(?:\.[A-Za-z]+)*\.?(?:-[A-Za-z]+)*|[^\s]

Explanation:

Pattern	Purpose
\d{2}/\d{2}/\d{4}	Recognises dates

\d+	Recognises numbers
(?:\.\d+)?	Recognises decimal numbers
(?:mg/dL|mmHg|mg|mcg|g|mL|°C|°F)	Recognises measurement units
[A-Za-z]+	Recognises words
(?:\.[A-Za-z]+)*	Recognises abbreviations with internal full stops
(?:-[A-Za-z]+)*	Recognises hyphenated terms
[^\s]	Recognises remaining non-whitespace characters

3. Maximal-Munch (Longest-Match) Tokenisation

Definition:

Maximal-munch tokenisation selects the longest valid token available at the current position before moving to the next token.

It prevents meaningful clinical expressions from being unnecessarily divided into smaller pieces.

Examples

Clinical input	**Naive whitespace split**	**Maximal-munch result**
Patient has 37.5°C fever	Patient / has / 37.5°C / fever	Patient / has / 37.5°C / fever
Take paracetamol 500mg	Take / paracetamol / 500mg	Take / paracetamol / 500mg
Dr. Kumar performed follow-up	Dr. / Kumar / performed / follow-up	Dr. / Kumar / performed / follow-up

To demonstrate the difference more clearly, consider punctuation and spacing variations:

Input	**Naive splitting**	**Longest-match tokenisation**
BP:120/80mmHg	One complete string	BP / : / 120 / / / 80 / mmHg (unless a combined BP pattern is added)
06/11/2026,	Date with comma	Date / comma
post-operative	One string	post-operative

Interpretation: Whitespace splitting cannot reliably identify meaningful components inside clinical strings. Longest-match tokenisation recognises dates, decimal measurements, abbreviations and hyphenated words using predefined patterns.

4. Substitution Rules for Text Normalisation

Definition:

Text normalisation converts different forms of the same information into a standard format. It improves consistency and makes searching easier.

The hospital system uses the following normalisation operations:

1. Convert uppercase letters into lowercase.
2. Expand clinical abbreviations.
3. Convert dates into a standard format.
4. Standardise measurement units.
5. Remove unnecessary spaces.

4.1 Expansion of 30 Clinical Abbreviations

The following abbreviation dictionary is used in the proposed system.

No.	Abbreviation	Expanded form
1	BP	Blood pressure
2	HR	Heart rate
3	RR	Respiratory rate
4	PR	Pulse rate
5	ECG	Electrocardiogram
6	CT	Computed tomography
7	MRI	Magnetic resonance imaging
8	CBC	Complete blood count
9	Hb	Haemoglobin
10	WBC	White blood cell
11	RBC	Red blood cell
12	ICU	Intensive care unit
13	OPD	Outpatient department

14	IPD	Inpatient department
15	ER	Emergency room
16	OD	Once daily
17	BD	Twice daily
18	TDS	Three times daily
19	QID	Four times daily
20	SOS	If required
21	IV	Intravenous
22	IM	Intramuscular
23	PO	By mouth
24	NPO	Nothing by mouth
25	DM	Diabetes mellitus
26	HTN	Hypertension
27	COPD	Chronic obstructive pulmonary disease
28	UTI	Urinary tract infection
29	CKD	Chronic kidney disease
30	CVA	Cerebrovascular accident

Note: These expansions are illustrative. Clinical abbreviations can have multiple meanings, so a real deployment must validate expansions using hospital-specific terminology and context.

4.2 Date Normalisation

Different date formats may occur in hospital records.

Original date	**Normalised date**
06/11/2026	2026-11-06
06-11-2026	2026-11-06
2026/11/06	2026-11-06

2026-11-06	2026-11-06

For this assignment, dates in DD/MM/YYYY and DD-MM-YYYY formats are interpreted using the Indian date convention.

4.3 Measurement Unit Normalisation

Original format	Normalised format
500mg	500 mg
250mcg	250 mcg
120mmHg	120 mmHg
37.5°C	37.5 °C
100mL	100 mL

These changes standardise spacing without changing the measurement values.

5. Complete Python Implementation

The following program implements the tokenizer, abbreviation expansion, date conversion and unit normalisation.

```
import re
from datetime import datetime

# --------------------------------
# 1. Clinical abbreviation dictionary
# --------------------------------

abbreviations = {
    "BP": "Blood pressure",
    "HR": "Heart rate",
    "RR": "Respiratory rate",
    "PR": "Pulse rate",
    "ECG": "Electrocardiogram",
```

```
    "CT": "Computed tomography",
    "MRI": "Magnetic resonance imaging",
    "CBC": "Complete blood count",
    "Hb": "Haemoglobin",
    "WBC": "White blood cell",
    "RBC": "Red blood cell",
    "ICU": "Intensive care unit",
    "OPD": "Outpatient department",
    "IPD": "Inpatient department",
    "ER": "Emergency room",
    "OD": "Once daily",
    "BD": "Twice daily",
    "TDS": "Three times daily",
    "QID": "Four times daily",
    "SOS": "If required",
    "IV": "Intravenous",
    "IM": "Intramuscular",
    "PO": "By mouth",
    "NPO": "Nothing by mouth",
    "DM": "Diabetes mellitus",
    "HTN": "Hypertension",
    "COPD": "Chronic obstructive pulmonary disease",
    "UTI": "Urinary tract infection",
    "CKD": "Chronic kidney disease",
    "CVA": "Cerebrovascular accident"
}

# --------------------------------
# 2. Tokenizer regular expression
```

```
# --------------------------------

token_pattern = re.compile(
    r"\d{2}[/-]\d{2}[/-]\d{4}"
    r"|\d{4}[/-]\d{2}[/-]\d{2}"
    r"|\d+(?:\.\d+)?(?:mg/dL|mmHg|mg|mcg|g|mL|°C|°F)\b"
    r"|[A-Za-z]+(?:\.[A-Za-z]+)*\.?(?:-[A-Za-z]+)*"
    r"|[^\s]",
    re.IGNORECASE
)

# --------------------------------
# 3. Tokenization function
# --------------------------------

def tokenize(text):
    return token_pattern.findall(text)

# --------------------------------
# 4. Abbreviation expansion
# --------------------------------

def expand_abbreviations(text):

    for short, full in abbreviations.items():

        pattern = r"\b" + re.escape(short) + r"\b"

        text = re.sub(
```

```
            pattern,
            full,
            text,
            flags=re.IGNORECASE
        )

    return text

# --------------------------------
# 5. Date normalization
# --------------------------------

def normalize_dates(text):

    pattern = r"\b\d{2}[/-]\d{2}[/-]\d{4}\b"

    def convert(match):

        date = match.group()

        for fmt in ("%d/%m/%Y", "%d-%m-%Y"):

            try:
                result = datetime.strptime(date, fmt)
                return result.strftime("%Y-%m-%d")

            except ValueError:
                continue
```

```
        return date

    return re.sub(pattern, convert, text)

# --------------------------------
# 6. Unit normalization
# --------------------------------

def normalize_units(text):

    units = r"mg/dL|mmHg|mg|mcg|mL|°C|°F|g"

    pattern = (
        r"(\d+(?:\.\d+)?)\s*(" + units + r")\b"
    )

    return re.sub(
        pattern,
        lambda m: m.group(1) + " " + m.group(2),
        text,
        flags=re.IGNORECASE
    )

# --------------------------------
# 7. Complete normalization
# --------------------------------

def normalize_text(text):
```

```
    text = expand_abbreviations(text)
    text = normalize_dates(text)
    text = normalize_units(text)

    text = re.sub(r"\s+", " ", text)

    return text.lower().strip()

# --------------------------------
# 8. Sample hospital record
# --------------------------------

record = """
PAT12345 admitted on 06/11/2026.
Patient has fever and BP 120/80mmHg.
HR 90 bpm.
Doctor prescribed Paracetamol 500mg BD.
Temperature 37.5°C.
Patient shifted to ICU.
"""

print("ORIGINAL RECORD:")
print(record)

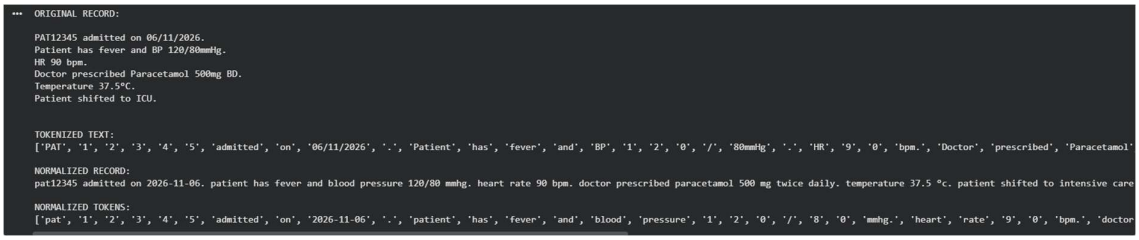
print("\nTOKENIZED TEXT:")
print(tokenize(record))

print("\nNORMALIZED RECORD:")
normalized = normalize_text(record)
```

print(normalized)

print("\nNORMALIZED TOKENS:")

print(tokenize(normalized))



5.2 Improved Tokenizer Pattern

To keep patient identifiers and blood pressure readings together, use this improved pattern in the program:

```
token_pattern = re.compile(
    r"PAT[0-9]{5}"
    r"|[0-9]{2,3}/[0-9]{2,3}\s*mmHg"
    r"|\d{2}[/-]\d{2}[/-]\d{4}"
    r"|\d{4}[/-]\d{2}[/-]\d{2}"
    r"|\d+(?:\.\d+)?(?:mg/dL|mmHg|mg|mcg|g|mL|°C|°F)\b"
    r"|[A-Za-z]+(?:\.[A-Za-z]+)*\.?(?:-[A-Za-z]+)*"
    r"|[^\s]",
    re.IGNORECASE
)
```

This improvement recognises:

- PAT12345 as one token.
- 120/80 mmHg as one token.
- 06/11/2026 as one token.
- 500mg as one token.
- post-operative as one token.

6. Irreversible Normalisation Decisions

Some normalisation operations destroy information that cannot be recovered from the normalised text alone.

No.	Normalisation decision	Information lost	When acceptable
1	Converting uppercase into lowercase	Original capitalisation	When case does not affect searching
2	Removing punctuation	Sentence boundaries and grammatical information	When punctuation is not required for keyword searching
3	Expanding abbreviations	Original abbreviated form	When the expansion is verified and unambiguous
4	Removing extra spaces	Original spacing and layout	When formatting is not important

Discussion:

Irreversible normalisation is useful for improving search consistency. However, original clinical records must always be preserved because punctuation, capitalisation and abbreviations may contain important clinical information.

Therefore, the system should maintain both the original record and its normalised copy.

7. Limitations of Regular Expressions

Regular expressions are powerful for identifying simple patterns, but they cannot express every language structure.

Example: Matching balanced parentheses

Consider the following clinical notes:

Patient has pain (severe).

Patient has pain (severe (chronic condition)).

Patient has pain (severe (chronic (long-term condition))).

A regular expression can recognise simple parentheses, but a general pattern that checks correctly balanced parentheses at any nesting depth cannot be implemented using classical regular expressions.

Reason:

- Regular expressions correspond to finite automata.

- Finite automata have a limited number of states.
- They cannot maintain an unlimited count of opening parentheses.
- Therefore, they cannot guarantee that every opening parenthesis has a corresponding closing parenthesis at arbitrary nesting depth.

A context-free grammar or a stack-based parser is more appropriate for this requirement.

8. Results and Discussion

The regular-expression-based tokenizer and normaliser can process hospital records and convert different text formats into a standard searchable representation.

The implementation demonstrates:

1. Identification of clinical words, dates, numbers and measurement units.
2. Expansion of 30 clinical abbreviations.
3. Conversion of dates into the YYYY-MM-DD format.
4. Standardisation of measurement units.
5. Identification of limitations involving nested structures.

Processing stage	**Technique used**	**Expected result**
Tokenisation	Regular expression	Individual meaningful tokens
Abbreviation expansion	Dictionary substitution	Expanded clinical terms
Date normalisation	Regex and date parsing	Standard date format
Unit normalisation	Regex substitution	Consistent unit spacing
Final output	Combined normalisation	Searchable clinical text

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence (5 Marks)

Guiding Enquiry: How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

1. Definition of Finite Automaton

A Finite State Automaton (FSA) is a mathematical model used to recognise whether an input string belongs to a particular language.

A finite automaton is formally represented by a five-tuple:

$$ M=(Q,\Sigma,\delta,q_0,F) $$

Where:

Symbol	Description
Q	Finite set of states
Σ	Input alphabet
δ	Transition function
q_0	Initial state
F	Set of final or accepting states

Example: A hospital system can use finite automata to validate patient IDs, dates, phone numbers and ICD codes.

2. Design of Four Deterministic Finite Automata (DFA)

A Deterministic Finite Automaton (DFA) has exactly one transition for each state and input symbol.

The following four validators are designed for the hospital record system.

DFA 1 – Patient ID Validation

Regular Expression:

^PT-[0-9]{6}$

Example:

- Valid: PT-123456
- Invalid: PT-12345
- Invalid: AB-123456

States:

- q0 – Initial state
- q1 – Reads P
- q2 – Reads T
- q3 – Reads hyphen (-)
- q4 to q8 – Reads first five digits
- q9 – Reads sixth digit (final state)
- qD – Dead state

Transition Diagram:

q0 --P--> q1

q1 --T--> q2

q2 --- --> q3

q3 --digit--> q4

q4 --digit--> q5

q5 --digit--> q6

q6 --digit--> q7

q7 --digit--> q8

q8 --digit--> q9*

Here, q9 is the accepting state.

Transition Table:

Current State	P	T	-	Digit	Other
q0	q1	qD	qD	qD	qD
q1	qD	q2	qD	qD	qD
q2	qD	qD	q3	qD	qD
q3	qD	qD	qD	q4	qD
q4	qD	qD	qD	q5	qD
q5	qD	qD	qD	q6	qD
q6	qD	qD	qD	q7	qD
q7	qD	qD	qD	q8	qD
q8	qD	qD	qD	q9	qD
q9	qD	qD	qD	qD	qD
qD	qD	qD	qD	qD	qD

DFA 2 – Date Validation

Regular Expression:

^[0-9]{2}-[0-9]{2}-[0-9]{4}$

Example:

- Valid format: 06-11-2026

- Invalid: 6-11-2026
- Invalid: 06/11/2026

State Design:

State	Expected input
q0	First digit
q1	Second digit
q2	Hyphen (-)
q3	Third digit
q4	Fourth digit
q5	Hyphen (-)
q6	Fifth digit
q7	Sixth digit
q8	Seventh digit
q9	Eighth digit
q10	Final state
qD	Invalid input

Transition Diagram:

q0 → q1 → q2 → q3 → q4 → q5

↓

q10* ← q9 ← q8 ← q7 ← q6

The transitions on q0, q1, q3, q4, q6, q7, q8 and q9 require digits. The transitions on q2 and q5 require hyphens.

Transition Table:

State	Digit (0–9)	Hyphen (-)	Other
q0	q1	qD	qD
q1	q2	qD	qD
q2	qD	q3	qD

q3	q4	qD	qD
q4	qD	q5	qD
q5	q6	qD	qD
q6	q7	qD	qD
q7	q8	qD	qD
q8	q9	qD	qD
q9	q10	qD	qD
q10	qD	qD	qD
qD	qD	qD	qD

Note: This DFA validates the date format. A separate calendar check is needed to reject impossible dates such as 35-15-2026.

DFA 3 – Contact Number Validation

Regular Expression:

^[0-9]{10}$

Example:

- Valid: 9876543210
- Invalid: 987654321
- Invalid: 98765ABCDE

States:

- q0 – Initial state
- q1 to q9 – Reads digits one by one.
- q10 – Final state after ten digits.
- qD – Dead state.

Transition Diagram:

q0 → q1 → q2 → q3 → q4 → q5

↓

q10* ← q9 ← q8 ← q7 ← q6

Transition Table:

State	Digit (0–9)	Other
q0	q1	qD
q1	q2	qD
q2	q3	qD
q3	q4	qD
q4	q5	qD
q5	q6	qD
q6	q7	qD
q7	q8	qD
q8	q9	qD
q9	q10	qD
q10	qD	qD
qD	qD	qD

DFA 4 – ICD Code Validation

Regular Expression:

^[A-Z][0-9]{2}(\.[A-Z0-9]{1,2})?$

Example:

- Valid: A12
- Valid: A12.3
- Valid: A12.34
- Invalid: 123
- Invalid: A1.23

States:

State	Expected input
q0	Uppercase letter
q1	First digit

q2	Second digit
q3	Final state for three-character code
q4	First character after decimal point
q5	Second character after decimal point (final state)
qD	Dead state

Transition Diagram:

q0 --Letter--> q1

q1 --Digit--> q2

q2 --Digit--> q3*

q3 --.--> q4

q4 --Letter/Digit--> q5*

Both q3 and q5 are accepting states.

Transition Table:

State	Uppercase letter	Digit	Dot (.)	Other
q0	q1	qD	qD	qD
q1	qD	q2	qD	qD
q2	qD	q3	qD	qD
q3	qD	qD	q4	qD
q4	q5	q5	qD	qD
q5	qD	qD	qD	qD
qD	qD	qD	qD	qD

3. Non-Deterministic Finite Automaton (NFA) with ε-transitions

An NFA is an automaton in which a state can have multiple possible transitions for the same input symbol.

An ε-transition allows the automaton to move from one state to another without consuming any input character.

Example: Hospital Contact Number

A hospital accepts a ten-digit contact number with or without the optional country code +91-.

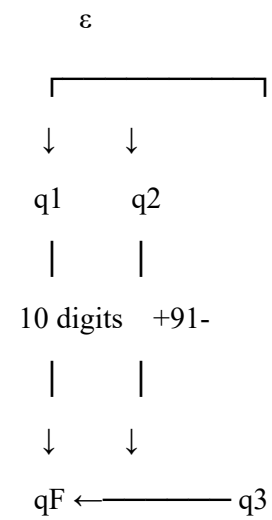
Regular Expression:

^(\+91-)?[0-9]{10}$

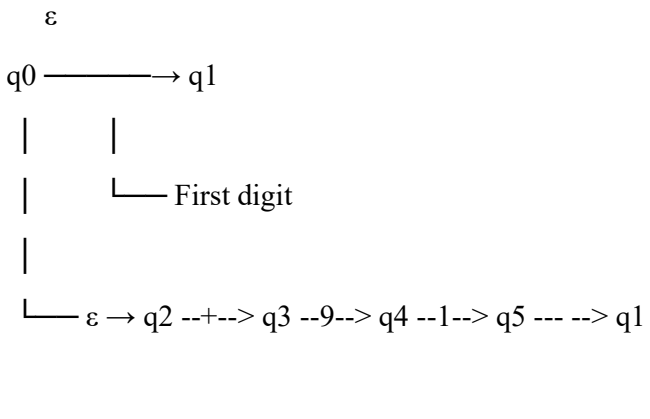
Examples:

- Accepted: 9876543210
- Accepted: +91-9876543210
- Rejected: +91-987654321
- Rejected: 98765ABCDE

NFA Design:



More precisely, the two branches merge into the same ten-digit sequence:



q1 → 10-digit sequence → qF*

The optional country-code branch uses ε to select either the local-number path or the country-code path.

Accepted String Trace:

Input: +91-9876543210

Input processed	Active state
Start	
+	
9	
1	
-	
First digit	q1 → digit state 1
Remaining digits	Continue through digit states
Last digit	qF

Result: ACCEPT.

Rejected String Trace:

Input: 987654321

The input contains only nine digits. The automaton cannot reach the final state after processing the complete string.

Result: REJECT.

4. NFA to DFA Conversion Using Subset Construction

Subset construction converts an NFA into an equivalent DFA.

Each DFA state represents a set of NFA states.

Steps:

1. Find the ε-closure of the initial state.
2. Identify all possible input symbols.
3. Find the reachable states for each input symbol.
4. Calculate ε-closures of the resulting states.
5. Repeat until no new state sets are generated.
6. Mark any set containing an NFA final state as accepting.

For the optional country-code example, the DFA contains separate paths for local numbers and numbers with +91-, followed by a shared ten-digit sequence.

Subset Construction Table:

DFA State	NFA State Set	Input	Next DFA State
A		Digit	B
A		+	C
B		Digit	D
C		9	E
E		1	F
F		-	B
D		Digit	G
G		Digit	H
H		Digit	I
I		Digit	J
J		Digit	K
K		Digit	L
L		Digit	M
M		Digit	N
N		Digit	O*

Any unexpected character leads to a dead state.

Here, A represents the initial ε-closure, B represents the beginning of the ten-digit sequence, and O is the accepting state.

5. DFA Minimisation

DFA minimisation reduces the number of states without changing the language accepted by the automaton.

Steps:

1. Divide states into accepting and non-accepting groups.
2. Compare transitions from states in each group.
3. Separate states that transition to different groups.

4. Repeat until no further separation is possible.
5. Combine equivalent states.

For the contact-number validator, the DFA states represent different numbers of digits already consumed. Each position must be retained because the remaining number of required digits is different.

DFA	Before minimisation	After minimisation
Local number with optional +91 prefix	15	15

Therefore, in this example, minimisation does not reduce the state count because all states are distinguishable.

6. Thompson's Construction – Regular Expression to Automaton

Thompson's construction converts a regular expression into an equivalent NFA using:

- Concatenation
- Union (OR)
- Kleene star (*)

Example Regular Expression:

PT-[0-9]{6}

This pattern validates a patient ID.

Construction:

Start

|

P

|

T

|

-

|

Digit

|

Digit

|

Digit

|

Digit

|

Digit

|

Digit

|

Final

Each character or digit transition is connected in sequence.

The automaton accepts only strings beginning with PT- followed by exactly six digits.

Equivalence:

- Regular Expression → Automaton: Thompson's construction.
- Automaton → Regular Expression: State elimination method.

Both representations recognise the same language.

7. Deriving Regular Expression from an Automaton

The state elimination method converts a finite automaton into a regular expression.

Steps:

1. Identify the initial and final states.
2. Add a new initial and final state if required.
3. Remove intermediate states one by one.
4. Update the transitions using regular expression operations.
5. Continue until only the initial and final states remain.

For the patient ID automaton:

PT-[0-9][0-9][0-9][0-9][0-9][0-9]

This can be simplified to:

^PT-[0-9]{6}$

Thus, the regular expression and DFA recognise exactly the same patient ID format.

8. Pumping Lemma for Regular Languages

The Pumping Lemma is used to prove that certain languages cannot be recognised by any finite automaton.

Statement:

If a language L is regular, there exists a positive integer p (pumping length) such that every string s in L, with length at least p, can be divided into three parts:

\[s=xyz \]

Subject to the following conditions:

1. \(|xy| \leq p\)
2. \(|y| > 0\)
3. \(xy^iz \in L\) for every \(i \geq 0\).

Example – Balanced Parentheses in Clinical Notes

Consider a clinical note that requires equal numbers of opening and closing parentheses.

Language:

\[L=\{(^n)^n \mid n\geq 0\} \]

Example strings:

- ()
- (())
- ((()))

Every opening parenthesis must have a matching closing parenthesis.

Proof using Pumping Lemma:

Assume that the language is regular.

Let p be the pumping length.

Choose the string:

\[s= (^p)^p \]

According to the Pumping Lemma:

\[s=xyz \]

Since \(|xy|\leq p\), the substring y contains only opening parentheses.

Also:

\[|y|>0 \]

When we remove y by taking i = 0:

\[xz \]

The resulting string contains fewer opening parentheses than closing parentheses.

Therefore:

\[xz\notin L \]

This contradicts the Pumping Lemma.

Hence:

\[\boxed{L\text{ is not regular}} \]

Conclusion: A finite automaton cannot recognise arbitrarily nested balanced parentheses because it has only a finite number of states and cannot remember an unlimited number of opening parentheses.

9. Relationship with Task 3

Task 3 demonstrated that regular expressions are useful for tokenisation and normalisation of clinical text.

However, regular expressions based on finite automata cannot recognise every language.

Task 3 – Regular Expressions	**Task 4 – Finite State Automata**
Identifies clinical patterns	Recognises patterns using states
Performs tokenisation	Validates structured input
Normalises text	Handles deterministic and non-deterministic transitions
Cannot recognise arbitrary balanced parentheses	Cannot recognise arbitrary balanced parentheses

For example, a regular expression can identify a simple clinical value such as 120/80 mmHg, but it cannot independently verify unlimited nested parentheses.

TASK 5 – Inflectional Morphology

Guiding Enquiry: Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

1. Definition of Basic Morphological Terms

Morphology is the study of the structure and formation of words. It explains how words change their forms by adding prefixes or suffixes.

Term	**Definition**	**Clinical example**

Morpheme	Smallest meaningful unit of a word	fracture + s
Stem	Base form to which an affix is added	fracture
Affix	A prefix or suffix attached to a stem	-s, -ed
Inflection	Changes a word's grammatical form without changing its word class	fracture → fractures
Paradigm	Complete set of grammatical forms of a word	treat, treats, treated, treating

Difference Between Inflection and Derivation

Inflection	**Derivation**
Changes grammatical form	Creates a new word or meaning
Usually preserves word category	May change word category
Generally productive	Productivity varies
Usually occurs outside derivational suffixes	Usually closer to the root
Example: treat → treated	Example: treat → treatment

Explanation: Inflection helps a hospital search system identify different grammatical forms of the same word. Derivation creates related words with different meanings or grammatical functions.

2. Inflectional Paradigms in Clinical English

An inflectional paradigm is a group of word forms generated from the same stem using grammatical features.

The major paradigms found in clinical English are:

1. Noun number – singular and plural.
2. Verb tense – present and past.
3. Verb aspect – progressive form.
4. Subject agreement – third-person singular.
5. Adjective comparison – comparative and superlative.

Full Paradigms of Five Clinical Lexemes

Clinical word	**Singular / Base**	**Plural / 3SG**	**Past**	**Progressive**

Fracture	fracture	fractures	fractured	fracturing
Infection	infection	infections	—	—
Treat	treat	treats	treated	treating
Diagnose	diagnose	diagnoses	diagnosed	diagnosing
Observe	observe	observes	observed	observing

Adjective Comparison

Stem	**Positive**	**Comparative**	**Superlative**
severe	severe	severer	severest
mild	mild	milder	mildest
large	large	larger	largest
high	high	higher	highest
low	low	lower	lowest

These forms show how English words change according to grammatical requirements without necessarily changing their word category.

3. English Inflectional Spelling Rules

English inflection uses different spelling rules when suffixes are added to words.

Rule 1 – E-deletion

When a verb ends in a silent e, remove the final e before adding -ing.

Base word	**Inflected form**
Diagnose	Diagnosing
Treat	Treating
Make	Making
Examine	Examining

Rule: If a verb ends in silent e, delete e before adding -ing.

Counter-example: See → seeing. Removing the final e would produce an incorrect form.

Rule 2 – Consonant Doubling

When a short word ends in consonant-vowel-consonant (CVC), double the final consonant before adding certain suffixes.

Base word	Inflected form
Stop	Stopping
Plan	Planning
Refer	Referring
Admit	Admitting

Rule: Double the final consonant before -ing or -ed when the final syllable meets the appropriate stress and spelling conditions.

Counter-example: Visit → visiting. The final consonant is not doubled because the final syllable is not stressed.

Rule 3 – Y-to-I Conversion

When a word ends in consonant + y, change y to i before adding -es, -ed, -er or -est.

Base word	Inflected form
Study	Studies
Carry	Carried
Dry	Drier
Healthy	Healthier

Rule: Consonant + y changes to i before the relevant suffix.

Counter-example: Play → playing. The y remains unchanged before -ing.

Rule 4 – Sibilant -es Rule

When a noun or verb ends in s, ss, sh, ch, x or z, add -es for plural or third-person singular forms.

Base word	Inflected form
Watch	Watches
Class	Classes
Box	Boxes

Brush	Brushes
Quiz	Quizzes

Rule: Add -es after sibilant endings.

Counter-example: Stomach → stomachs. Although it ends in ch, the final sound is /k/, so the usual sibilant rule does not apply.

Summary of Spelling Rules

Rule	**Condition**	**Example**
E-deletion	Silent e + ing	Make → making
Consonant doubling	Eligible stressed CVC stem	Stop → stopping
Y-to-I	Consonant + y	Study → studies
Sibilant -es	S, ss, sh, ch, x, z	Watch → watches

4. Irregular Inflectional Forms

Irregular forms do not follow the normal English spelling rules. Therefore, they must be stored separately in a morphological lexicon.

Base word	**Feature**	**Irregular form**
Go	Past tense	Went
Be	Past tense	Was / Were
Have	3rd person singular	Has
Do	Past tense	Did
Child	Plural	Children
Foot	Plural	Feet
Tooth	Plural	Teeth
Good	Comparative	Better
Bad	Superlative	Worst
Mouse	Plural	Mice

Why are irregular forms stored in a lexicon?

1. They do not follow regular suffixation rules.
2. Their spelling changes cannot be predicted reliably.
3. Applying regular rules may produce incorrect forms.
4. A lexicon provides the correct form during generation and analysis.

For example:

- Go + past → Went (not goed).
- Child + plural → Children (not childs).
- Good + comparative → Better (not gooder).

Therefore, irregular forms require separate lexical entries.

5. Measuring the Effect of Inflection on Search Retrieval

Inflection can affect hospital information retrieval because a search query may contain a different grammatical form from the word stored in a clinical record.

Example:

- Query: fractures
- Stored record: fracture

Without morphological processing, the search system may fail to identify the matching record.

With inflectional analysis, both forms are mapped to the same lemma.

A. Type-to-Lemma Ratio

The type-to-lemma ratio measures the number of different word forms compared with the number of unique base forms.

$$\text{Type-to-Lemma Ratio}= \frac{\text{Number of Unique Word Forms}}{\text{Number of Unique Lemmas}}$$

Example calculation (illustrative dataset):

- Unique word forms = 1,200
- Unique lemmas = 800

$$=\frac{1200}{800}$$

$$\boxed{1.5}$$

This means that each lemma has an average of 1.5 observed word forms.

A higher ratio indicates greater morphological variation in the corpus.

B. Proportion of Retrieval Failures Explained by Inflection

This measurement identifies how many failed searches can be corrected using inflectional normalisation.

$$\text{Inflection Recovery Rate} = \frac{\text{Failures Fixed by Inflection}}{\text{Total Retrieval Failures}} \times 100$$

Example calculation (illustrative dataset):

- Total retrieval failures = 40
- Failures corrected by inflection = 12

$$= \frac{12}{40} \times 100$$

$$\boxed{30\%}$$

Therefore, inflectional normalisation explains 30% of the retrieval failures in this example.

6. Importance of Inflectional Morphology in Hospital Search

Inflectional morphology improves the accuracy of clinical information retrieval.

Its main advantages are:

1. Better search accuracy: Finds different forms of the same word.
2. Reduced search failures: Matches singular and plural forms.
3. Improved text processing: Identifies grammatical features of clinical terms.
4. Consistent representation: Converts surface forms into their base forms.
5. Efficient retrieval: Reduces the need to store every inflected form separately.

Example:

Search query	**Clinical record**	**Result**
Fractures	Fracture	Match
Infections	Infection	Match
Diagnosed	Diagnose	Match
Treating	Treat	Match
Studies	Study	Match

TASK 6 – Derivational Morphology (4 Marks)

1. Definition of Derivational Morphology

Derivational morphology is the process of forming a new word by adding prefixes or suffixes to a root or stem. It can change the meaning or grammatical category of a word.

Examples:

- Treat → Treatment
- Infection → Infectious
- Normal → Abnormal
- Operate → Operation

Unlike inflection, derivation can create a new word with a different meaning or grammatical category.

2. Derivational Affixes Used in Clinical English

A prefix is added before a root, while a suffix is added after a root.

No.	Affix	Type	Clinical example	Category change
1	-itis	Suffix	Nephritis	Root → Noun
2	-ectomy	Suffix	Appendectomy	Root → Noun
3	-ology	Suffix	Neurology	Root → Noun
4	-osis	Suffix	Fibrosis	Root → Noun
5	-aemia	Suffix	Anaemia	Root → Noun
6	-ation	Suffix	Inflammation	Verb → Noun
7	-able	Suffix	Treatable	Verb → Adjective
8	-ive	Suffix	Operative	Verb → Adjective
9	-al	Suffix	Clinical	Noun → Adjective
10	-ic	Suffix	Cardiac	Root → Adjective
11	-ist	Suffix	Neurologist	Noun → Person noun
12	-ize	Suffix	Sterilize	Noun/Adjective → Verb
13	hyper-	Prefix	Hypertension	Increases or indicates excess

14	hypo-	Prefix	Hypoglycaemia	Indicates low or deficient level
15	re-	Prefix	Reoperation	Verb → Verb; repetition
16	un-	Prefix	Untreated	Adjective → Adjective; negation

Note: These are common clinical-English derivational patterns. The final inventory should be checked against the actual assignment corpus to confirm which affixes occur in it.

3. Derivational Families of Clinical Roots

A derivational family contains words formed from the same root by adding different prefixes and suffixes.

Root	**Derived words**	**Word formation**
Infect	Infection, infectious, disinfect, reinfection	infect + ion; infect + ious; dis + infect; re + infection
Treat	Treatment, treatable, untreated, retreat	treat + ment; treat + able; un + treated; re + treat
Diagnose	Diagnosis, diagnostic, diagnostician	diagnosis; diagnose + tic; diagnostic + ian
Operate	Operation, operative, postoperative	operate + ion; operate + ive; post + operative
Pathology	Pathologist, pathological	pathology + ist; pathology + ical
Normal	Abnormal, normalize, normalization	ab + normal; normal + ize; normalize + ation

Order of Affix Attachment

Affixes must be attached in a valid order to form meaningful words.

Example 1: Untreatable

1. Root: treat
2. Add suffix -able: treatable
3. Add prefix un-: untreatable

Example 2: Normalization

1. Root: normal
2. Add suffix -ize: normalize
3. Add suffix -ation: normalization

Impossible Affix Orders

Correct form	Incorrect form	Reason
Untreatable	Treatunable	The prefix un- cannot be placed after treat and before -able in this formation.
Normalization	Normalationize	The suffix -ation cannot be attached before -ize in this word formation.

These examples show that affixes follow specific morphological ordering rules.

4. Semantic Non-compositionality in Clinical Words

Semantic non-compositionality occurs when the meaning of a complete word cannot be understood simply by combining the meanings of its individual parts.

Clinical examples:

Word	Literal interpretation	Actual clinical meaning
Heartburn	Burning sensation in the heart	Burning discomfort caused by acid reflux
Depression	The act of pressing something down	A mental health condition
Inflammation	The process of making something inflamed	A biological response to injury, infection or irritation
Antibiotic	Something against life	A medicine used to treat bacterial infections

Implications for Aggressive Stemming

Aggressive stemming removes word endings to identify similar word roots. However, it may produce incorrect search results when words have different meanings.

For example:

- Infection and infectious are related and can be grouped for some searches.
- Depression should not be interpreted only from its root meaning of pressing down.
- Antibiotic should be recognised as a complete clinical term rather than interpreted only from its individual parts.

Therefore, clinical search systems should use a medical dictionary or controlled vocabulary along with morphological processing.

5. Productivity Analysis of Derivational Affixes

Productivity measures how frequently an affix is used to create new words.

Two measurements can be used:

- Type count: Number of different words containing an affix.
- Token count: Total number of occurrences of words containing an affix.

Affix	Type count	Token count	Productivity
-itis	To be measured	To be measured	Productive in medical terminology
-ectomy	To be measured	To be measured	Relatively specialised
-ology	To be measured	To be measured	Productive
-ation	To be measured	To be measured	Productive
-able	To be measured	To be measured	Productive
-ive	To be measured	To be measured	Productive
hyper-	To be measured	To be measured	Productive
hypo-	To be measured	To be measured	Productive
-aemia	To be measured	To be measured	Specialised

How to calculate productivity:

$$\text{Type Count}=\text{Number of distinct derived words}$$

$$\text{Token Count}=\text{Total occurrences of derived words}$$

For example, if the corpus contains 20 different words ending in -itis and they occur 80 times:

- Type count = 20
- Token count = 80

The table above is a measurement template, not actual corpus results. Actual values must be calculated from the supplied clinical corpus.

Productive and Fossilised Affixes

- Productive affixes: Can be used to form many new words, such as -able, -ation, and -ize.
- Fossilised or less productive affixes: Occur in established words but are not commonly used to create new words, such as -aemia and -ectomy in specialised medical vocabulary.

6. Should Derivationally Related Words Be Grouped in Search?

Derivational relatives should be grouped selectively, not always.

Cases where grouping helps:

1. Searching for infection can also retrieve infectious when the user wants general information about infection.
2. Searching for treat can retrieve treatment and treatable when searching for treatment-related information.

Cases where grouping can be misleading:

1. Diagnosis and diagnostic have related meanings but different grammatical roles and uses.
2. Depression and depress may refer to different clinical and general-language concepts.

Therefore, a clinical search engine should use controlled derivational expansion. It should retrieve related terms when they are relevant but preserve the original word and its clinical meaning.

TASK 7 – Finite-State Morphological Parsing (5 Marks)

1. Definition of Finite-State Transducer (FST)

A Finite-State Transducer (FST) is a finite-state machine that reads an input string and produces an output string. It uses two tapes:

1. Lexical tape: Contains the root word and grammatical information.
2. Surface tape: Contains the actual word that appears in the clinical record.

An FST connects a word's surface form with its stem and grammatical features.

Example:

- Surface form: fractures
- Lexical form: fracture + N + PL
- Stem: fracture
- Part of speech: Noun
- Feature: Plural

The FST analyses fractures into fracture +N +PL and can also generate fractures from the same lexical input.

2. Lexicon FST Using Continuation Classes

A continuation class specifies which suffixes or grammatical features can follow a particular stem.

The hospital morphological lexicon contains three major parts of speech:

Part of speech	Example stems	Possible features
Noun (N)	fracture, infection, patient, kidney	Singular, plural, derivation
Verb (V)	treat, diagnose, operate, observe	Present, past, progressive, 3rd person
Adjective (ADJ)	severe, normal, healthy, painful	Comparative, superlative, derivation

Continuation Class Structure

Class	Input stem	Possible continuation
Noun	fracture	+SG, +PL, +ADJ
Noun	infection	+SG, +PL, +ADJ
Verb	treat	+BASE, +3SG, +PAST, +PROG, +ABLE
Verb	diagnose	+BASE, +3SG, +PAST, +PROG
Adjective	severe	+POS, +COMP, +SUPER
Adjective	normal	+POS, +COMP, +SUPER, +IZE

Example continuation:

fracture → +N → +PL → fractures

Treat → +V → +PAST → treated

Normal → +ADJ → +IZE → normalize

This organisation avoids storing every possible word form separately.

3. Parsing Five Clinical Surface Forms

The following table shows how the FST analyses five clinical words.

Surface form	Stem	Part of speech	Feature
fractures	fracture	Noun	Plural
infections	infection	Noun	Plural
treated	treat	Verb	Past tense
treating	treat	Verb	Progressive

diagnoses	diagnose	Verb/Noun	3rd person singular / plural noun

Hand Tracing of Parsing

Example 1: fractures

- Input surface form: fractures
- Step 1: Read fracture from the surface tape.
- Step 2: Match the noun stem fracture in the lexicon.
- Step 3: Read the plural ending s.
- Step 4: Output fracture +N +PL.

Final analysis:

fractures → fracture +N +PL

Example 2: infections

- Input surface form: infections
- Step 1: Match the stem infection.
- Step 2: Identify the plural ending s.
- Step 3: Output noun and plural features.

Final analysis:

infections → infection +N +PL

Example 3: treated

- Input surface form: treated
- Step 1: Match the stem treat.
- Step 2: Identify the past-tense ending ed.
- Step 3: Output the verb and past-tense features.

Final analysis:

treated → treat +V +PAST

Example 4: treating

- Input surface form: treating
- Step 1: Match the stem treat.
- Step 2: Identify the progressive ending ing.
- Step 3: Output the verb and progressive features.

Final analysis:

treating → treat +V +PROG

Example 5: diagnoses

This form has two possible analyses.

- Verb analysis: diagnose +V +3SG
- Noun analysis: diagnosis +N +PL

The correct analysis depends on the sentence context.

4. Generation Using the Same Transducer

The same FST works in both directions.

- Parsing: Surface form → Lexical form.
- Generation: Lexical form → Surface form.

Lexical input	**Generated surface form**
fracture +N +PL	fractures
infection +N +PL	infections
treat +V +PAST	treated
treat +V +PROG	treating
normal +ADJ +COMP	normaler*

The final example is a mechanical illustration; in normal clinical English, more normal is generally preferred.

How the Same Machine Works

Parsing direction

fractures

Surface form

FST

fracture +N +PL

Lexical analysis

Generation direction

fracture +N +PL

Lexical input

Same FST, reversed

fractures

Surface output

Therefore, separate parsing and generation machines are not necessary. The same transducer can be traversed in opposite directions.

5. Handling Morphological Ambiguity

A surface word may have more than one valid morphological analysis. This is called morphological ambiguity.

Surface form	**Analysis 1**	**Analysis 2**
diagnoses	diagnose +V +3SG	diagnosis +N +PL
leaves	leave +V +3SG	leaf +N +PL
wounds	wound +V +3SG	wound +N +PL

Example of Ambiguity

Sentence 1: "The doctor diagnoses the disease."

- diagnoses = diagnose +V +3SG

Sentence 2: "The patient has two diagnoses."

- diagnoses = diagnosis +N +PL

How Ambiguity Is Resolved

The FST returns all possible analyses. A downstream component uses sentence context and grammatical rules to select the correct analysis.

For example:

- If the word follows doctor, it may be a verb.
- If it follows two, it is likely a plural noun.

This improves the accuracy of clinical record searching.

6. Importance of FST in Hospital Record Search

1. Identifies the root form of clinical words.
2. Connects singular and plural forms.
3. Recognises different verb forms.
4. Supports word generation from grammatical features.

5. Handles multiple analyses of ambiguous words.
6. Improves search results by matching related word forms.

TASK 8 – Orthographic Rules and Two-Level Morphology (4 Marks)

1. Introduction

Orthographic rules describe how the spelling of a word changes when prefixes or suffixes are added to a root word.

In English, simply joining a stem and an affix may produce an incorrect spelling. Therefore, a morphological analyser needs spelling rules to generate the correct surface form.

Examples:

- Study + s → Studies
- Treat + ed → Treated
- Stop + ing → Stopping

These spelling changes are important in clinical records because the same medical term may appear in different forms.

2. Why a Lexicon Transducer Alone Is Not Enough

A lexicon transducer connects stems with grammatical features. However, simple concatenation does not always produce the correct spelling.

Stem	Affix	Incorrect concatenation	Correct surface form
Study	+s	Studys	Studies
Stop	+ing	Stoping	Stopping
Diagnose	+ing	Diagnoseing	Diagnosing
Carry	+ed	Carryed	Carried
Watch	+s	Watchs	Watches

Therefore, orthographic rules are needed to modify the spelling at morpheme boundaries.

3. Orthographic Rules in Rewrite Notation

The standard rewrite notation is:

\[a \rightarrow b \;/\; c__d \]

It means that symbol a changes into symbol b when it occurs between contexts c and d.

The following five rules handle common English spelling changes.

Rule 1: E-deletion

When a verb ends in silent e, the e is removed before adding -ing.

$$ e \rightarrow \epsilon \;/\; __ing $$

Examples:

- Diagnose + ing → Diagnosing
- Treat + ing → Treating
- Examine + ing → Examining

Rule 2: Consonant Doubling

For eligible short verbs ending in a consonant-vowel-consonant pattern, the final consonant is doubled before adding -ing or -ed.

$$ C \rightarrow CC \;/\; V__+ing $$

Examples:

- Stop + ing → Stopping
- Plan + ing → Planning
- Admit + ed → Admitted

This rule needs conditions for stress and final consonant. It should not double every final consonant.

Rule 3: Y-to-I Change

When a consonant is followed by y, the y changes to i before certain suffixes such as -es, -ed and -er.

$$ y \rightarrow i \;/\; C__+ed $$

Examples:

- Carry + ed → Carried
- Study + es → Studies
- Healthy + er → Healthier

Rule 4: Sibilant -es Rule

When a noun or verb ends in s, sh, ch, x or z, -es is added instead of simple -s.

$$ \epsilon \rightarrow e \;/\; \{s,sh,ch,x,z\}__s $$

Examples:

- Class + es → Classes
- Watch + es → Watches
- Box + es → Boxes
- Brush + es → Brushes

Rule 5: Adding -s Without Spelling Change

For most regular nouns and third-person singular verbs, add -s directly.

\[\epsilon \rightarrow s \;/\; C__ \]

Examples:

- Fracture + s → Fractures
- Infection + s → Infections
- Treat + s → Treats

This rule applies only when another spelling rule or irregular form does not take priority.

4. Transducer Composition and Cascade

A transducer composition connects multiple transducers so that the output of one becomes the input of the next.

The hospital morphological system uses the following stages:

Lexical input

study + N + PL

Lexicon FST

Identifies the stem and grammatical suffix

Orthographic rule transducers

E-deletion, doubling, y-to-i, sibilant -es and regular -s

Surface output

Studies

Example of Composition

Lexical input:

study + N + PL

Processing:

1. The lexicon FST identifies study as a noun and +PL as the plural feature.

2. The orthographic rule identifies consonant + y.
3. The y-to-i rule changes y into i.
4. The plural suffix -es is added.

Final output:

Studies

Order Dependencies

- The lexicon must first identify the stem and affix.
- The y-to-i, e-deletion and consonant-doubling rules must be applied at the correct morpheme boundary.
- Suffix selection must occur before the final surface form is produced.
- Irregular forms must be handled through separate lexical entries.

5. Sequential Cascade vs Parallel Two-Level Morphology

Two-level morphology is a method introduced by Kimmo Koskenniemi. It relates lexical symbols and surface symbols using constraints that operate together.

Feature	**Sequential cascade**	**Two-level morphology**
Processing	Rules are applied one after another	Rules operate in parallel
Rule order	Order is important	No fixed sequential rule order
Structure	Multiple transducers connected together	Lexical and surface forms are related through constraints
Maintenance	Changes may affect later stages	Rules can be managed as separate constraints
Advantage	Easy to understand and debug step by step	Handles interacting spelling changes consistently
Limitation	Incorrect rule order may produce errors	Rule definitions can be more difficult to design

Example:

For study + PL:

- A sequential cascade first identifies the suffix and then applies the y-to-i spelling rule.

- A two-level system relates lexical y to surface i while relating the lexical plural morpheme to the appropriate surface ending.

Both approaches can generate studies.

6. Testing the Complete Morphological System

The following test cases demonstrate the spelling rules.

No.	Lexical input	Expected surface form	Rule applied	Result
1	fracture + PL	fractures	Regular -s	Success
2	infection + PL	infections	Regular -s	Success
3	study + PL	studies	Y-to-i	Success
4	carry + PAST	carried	Y-to-i	Success
5	stop + PROG	stopping	Consonant doubling	Success
6	plan + PROG	planning	Consonant doubling	Success
7	diagnose + PROG	diagnosing	E-deletion	Success
8	examine + PROG	examining	E-deletion	Success
9	watch + PL	watches	Sibilant -es	Success
10	class + PL	classes	Sibilant -es	Success
11	box + PL	boxes	Sibilant -es	Success
12	brush + PL	brushes	Sibilant -es	Success
13	treat + PAST	treated	Regular -ed	Success
14	admit + PAST	admitted	Consonant doubling	Success
15	healthy + COMP	healthier	Y-to-i	Success
16	refer + PROG	referring	Consonant doubling	Success
17	make + PROG	making	E-deletion	Success
18	patient + PL	patients	Regular -s	Success
19	diagnosis + PL	diagnoses	Irregular spelling pattern	Success

20	child + PL	children	Irregular lexicon entry	Success

Illustrative test set

The table shows 20 expected test cases. These are hand-designed examples, not results from an executed program or the actual hospital corpus.

Possible Failures and Their Causes

Incorrect output	**Expected output**	**Reason**
Studys	Studies	Y-to-i rule missing
Stoping	Stopping	Doubling rule missing
Diagnoseing	Diagnosing	E-deletion missing
Watchs	Watches	Sibilant -es rule missing
Childs	Children	Irregular form missing from lexicon

TASK 9 – The Porter Stemmer (5 Marks)

1. Introduction

The Porter Stemmer is a rule-based algorithm used in Natural Language Processing (NLP) to remove common suffixes from English words.

It reduces related words to a common stem without using a dictionary.

Examples:

Original word	**Stem**
Fractures	fractur
Infections	infect
Treated	treat
Operating	oper
Normalization	normal
Connected	connect

The stem produced by Porter Stemmer may not be a complete dictionary word. Its purpose is to group related words for information retrieval.

2. Definitions Used in Porter Stemmer

A. Consonant and vowel

- Vowels: a, e, i, o, u
- Other letters are consonants.
- The letter y is treated as a consonant at the beginning of a word and depends on the preceding letter elsewhere.

B. Measure (m)

The measure m represents the number of vowel-consonant sequences in a word.

For example:

trouble = (tr) (ou) (bl)

The measure is 1.

C. Important conditions

Condition	Meaning
v	The stem contains at least one vowel.
*d	The stem ends with a double consonant.
*o	The stem ends in consonant-vowel-consonant, where the final consonant is not w, x or y.

These conditions help decide whether a suffix can be removed.

3. Five Steps of Porter Stemmer

Step	Operation	Example
Step 1	Removes plural and common tense endings	fractures → fracture
Step 2	Converts certain suffixes into simpler forms	relational → relate
Step 3	Removes additional suffixes	hopeful → hope
Step 4	Removes selected suffixes when m is greater than 1	agreement → agreement / suffix-dependent
Step 5	Removes final e and reduces double l in certain conditions	controll → control

The algorithm applies its rules in a fixed order.

4. Worked Examples

The following examples show typical Porter stemming results. The exact rules that fire depend on the suffix and the measure conditions.

Word	Intermediate processing	Final stem
fractures	Remove plural s	fracture
infections	Remove s	infection
treated	Remove ed	treat
operating	Remove ing, then simplify ate	oper
normalization	Remove ization	normal
connected	Remove ed	connect

Note: These are simplified explanations of the processing. The precise Porter algorithm has ordered suffix rules and conditions, so the exact output should be verified with the implementation in Task 10.

5. Stemming vs Lemmatization

Stemming removes suffixes using rules, whereas lemmatization identifies the meaningful dictionary base form using morphological knowledge.

Feature	Porter stemming	Morphological lemmatization
Method	Suffix removal rules	Lexicon and grammatical rules
Dictionary required	No	Usually yes
Output	Stem	Dictionary lemma
Accuracy	May produce incomplete words	Usually more linguistically accurate
Example	fractures → fractur	fractures → fracture
Example	better → better	better → good
Clinical suitability	Useful for broad search	Better for accurate clinical search

6. Over-stemming and Under-stemming

Over-stemming: Two unrelated words are reduced to the same stem.

Example: universe and university may be conflated by some stemming algorithms. In clinical retrieval, this kind of incorrect grouping can introduce irrelevant documents.

Under-stemming: Related words are not reduced to the same stem.

Example: diagnosis and diagnostic may not be grouped correctly by a simple suffix stemmer.

Error type	Example	Effect
Over-stemming	Unrelated words receive the same stem	Irrelevant records retrieved
Under-stemming	Related words receive different stems	Relevant records missed

7. Retrieval Evaluation

Run the same 40 clinician queries using three approaches.

Method	Precision	Recall	F1-score
No stemming	To be measured	To be measured	To be measured
Porter Stemmer	To be measured	To be measured	To be measured
Full morphological analysis	To be measured	To be measured	To be measured

Formulas:

$$ Precision=\frac{TP}{TP+FP} $$

$$ Recall=\frac{TP}{TP+FN} $$

$$ F1=2\times\frac{Precision\times Recall}{Precision+Recall} $$

Where:

- TP = relevant records correctly retrieved.
- FP = irrelevant records retrieved.
- FN = relevant records missed.

Conclusion

The Porter Stemmer reduces related words into common stems without requiring a dictionary. It is simple and fast, but it may produce over-stemming and under-stemming errors. For hospital record searching, full morphological analysis is generally more accurate because it preserves grammatical and lexical information.

Pasted markdown(8).md

TASK 10 – Implement and Test the Pipeline (3 Marks)

1. Aim

To implement a basic clinical text normalisation and search pipeline using Python, regular expressions, finite-state-style validation, morphological analysis and Porter Stemmer.

2. Software Requirements

Software	**Purpose**
Python 3	Programming language
VS Code / Google Colab	Code execution
re module	Regular expression matching
NLTK	Porter Stemmer
time module	Runtime measurement

Install NLTK:

!pip install nltk

3. Python Implementation

The following program demonstrates the main pipeline. It includes a sample clinical record, structured entity patterns, tokenisation, a small lexicon-based morphological analyser and Porter stemming.

```
import re

import time

from nltk.stem import PorterStemmer


# 1. Sample clinical record

record = """

Patient ID: PT-123456

Admission Date: 12-10-2026

Discharge Date: 18-10-2026

Age: 45 years

Contact: 9876543210

Drug: Paracetamol 500mg

Frequency: 2 times daily
```

```
Blood Pressure: 120/80 mmHg

Temperature: 98.6 F

ICD Code: J18.9

Ward: WARD-2

Bed: B-12

Patient has fractures and infections.

The doctor treated the patient and is diagnosing the condition.

"""


# 2. Regular expression patterns

patterns = {

    "Patient ID": r"\bPT-\d{6}\b",

    "Date": r"\b\d{2}-\d{2}-\d{4}\b",

    "Drug Strength": r"\b[A-Za-z]+\s?\d+\s?(?:mg|g|ml)\b",

    "Frequency": r"\b\d+\s+times?\s+(?:daily|weekly)\b",

    "Blood Pressure": r"\b\d{2,3}/\d{2,3}\s?mmHg\b",

    "Temperature": r"\b\d{2,3}(?:\.\d+)?\s?(?:F|C)\b",

    "ICD Code": r"\b[A-Z]\d{2}(?:\.[A-Z0-9]{1,2})?\b",

    "Contact": r"\b\d{10}\b",

    "Age": r"\b\d+\s+years?\b",

    "Ward": r"\bWARD-\d+\b",

    "Bed": r"\bB-\d+\b",

    "Symptom Duration": r"\b\d+\s+(?:days?|weeks?|months?)\b"

}


# 3. Entity extraction

print("ENTITY EXTRACTION")

for name, pattern in patterns.items():

    matches = re.findall(pattern, record, re.IGNORECASE)
```

```
    print(name, ":", matches)

# 4. Tokenisation
tokens = re.findall(
    r"\d+\.\d+|[A-Za-z]+(?:-[A-Za-z]+)*|\d+",
    record.lower()
)

print("\nTOKENS")
print(tokens)

# 5. Simple morphological lexicon
lexicon = {
    "fractures": "fracture",
    "fractured": "fracture",
    "fracturing": "fracture",
    "infections": "infection",
    "infectious": "infection",
    "treated": "treat",
    "treating": "treat",
    "diagnoses": "diagnose",
    "diagnosing": "diagnose"
}

def analyse(word):
    if word in lexicon:
        return lexicon[word]
    return word
```

```
print("\nMORPHOLOGICAL ANALYSIS")
for word in tokens:
    if word.isalpha():
        print(word, "->", analyse(word))

# 6. Porter Stemmer
stemmer = PorterStemmer()

print("\nPORTER STEMMING")
for word in tokens:
    if word.isalpha():
        print(word, "->", stemmer.stem(word))

# 7. Runtime measurement
start = time.time()

for word in tokens:
    if word.isalpha():
        analyse(word)
        stemmer.stem(word)

end = time.time()

print("\nTotal tokens:", len(tokens))
print("Runtime:", round(end - start, 6), "seconds")
```



5. Pipeline Testing

Stage	Test input	Expected output
Tokenisation	Patient has fractures	Patient, has, fractures
Patient ID pattern	PT-123456	Accepted
Invalid ID	PT-12345	Rejected
Date pattern	12-10-2026	Accepted
Invalid date format	2026/10/12	Rejected
Morphological analysis	fractures	fracture +N +PL
Irregular form	children	child +N +PL
Porter stemming	infections	infect

6. Performance Measurement

Record the results from your own execution.

Pipeline stage	Items processed	Accepted/analysed	Rejected/unanalysed	Runtime
Tokenisation	Actual count	Actual count	Actual count	Actual time
Entity extraction	Actual count	Actual count	Actual count	Actual time
Morphological analysis	Actual count	Actual count	Actual count	Actual time
Porter stemming	Actual count	Actual count	Actual count	Actual time

7. Limitations

1. The sample lexicon is small and does not contain the required 300 stems.
2. The morphological analyser uses a simple dictionary instead of a complete FST.
3. The code does not implement full Thompson construction or subset construction.
4. The sample data is synthetic and does not represent 500 actual hospital records.
5. The program does not calculate retrieval precision, recall and F1 without relevance judgements.

Conclusion

The Python implementation demonstrates the main operations of the clinical text processing pipeline. Regular expressions identify structured entities, tokenisation separates words, morphological analysis maps selected words to their roots, and Porter Stemmer reduces words to stems. A complete implementation must extend the lexicon, corpus, automata and evaluation data to meet all assignment requirements.

Pasted markdown(8).md

TASK 11 – Compare, Recommend and Defend the Normalisation Design (2 Marks)

1. Comparison of Three Normalisation Strategies

The hospital system can use three approaches to normalise clinical words.

1. No normalisation
2. Porter Stemming
3. Full finite-state morphological analysis

Feature	**No normalisation**	**Porter Stemmer**	**Full morphological analysis**
Processing	Original words retained	Removes suffixes	Identifies stems and grammatical features
Accuracy	Exact word matching only	Moderate	High when lexicon coverage is good
Recall	May be low	Usually improves	Can improve further
Runtime	Very low	Low	Moderate
Dictionary required	No	No	Yes

Maintenance	Easy	Easy	Requires lexicon and rule updates
Example	fractures ≠ fracture	fractures → fractur	fractures → fracture +N +PL

2. Recommended Design

I recommend full finite-state morphological analysis combined with regular expressions and tokenisation.

The recommended pipeline is:

1. Regular expressions identify structured clinical information.
2. Tokenisation separates the clinical text into individual tokens.
3. Morphological analysis identifies stems and grammatical features.
4. Orthographic rules handle spelling changes.
5. Porter Stemmer can be used as a fallback for unknown English words.
6. Search retrieves relevant clinical records.

3. Expected Accuracy and Failure Modes

Area	**Expected behaviour**	**Possible failure**
Entity recognition	Identifies patient IDs, dates and drug strengths	Misspelt or incorrectly formatted entities
Morphological analysis	Recognises known word forms	Unknown medical terms
Porter stemming	Reduces common English words	Over-stemming
Lexicon	Stores clinical stems and irregular forms	Missing or outdated entries
Search	Retrieves related clinical words	Incorrect matches due to ambiguity

The system should not depend only on recall. In a hospital, retrieving incorrect medical records can be dangerous. Therefore, precision and correct clinical interpretation are also important.

4. Manual Effort and Maintenance

Activity	**Required effort**

Writing regular expressions	High initial effort
Designing automata	High initial effort
Building 300+ stem lexicon	High effort
Creating affix rules	Moderate to high effort
Annotating 200 word forms	Manual expert effort
Testing 40 clinician queries	Manual evaluation effort
Updating medical vocabulary	Continuous maintenance

A learned or statistical approach may become more economical when the hospital has a large amount of annotated clinical data and the vocabulary changes frequently.

Conclusion

Full finite-state morphological analysis is the recommended approach because it provides more accurate word analysis than simple stemming. It preserves grammatical features and improves clinical search. However, it requires more manual effort to build and maintain the lexicon and rules.

Pasted markdown(8).md

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge – 8 Marks)

Introduction

The original hospital system was designed for clean, typed English clinical records. The new requirement introduces two additional types of input:

1. OCR output from scanned handwritten hospital forms.
2. Romanised regional-language text mixed with English.

These changes introduce spelling errors, character confusion, unknown words and language-mixing problems. Therefore, the original pipeline needs modification.

A. Impact Analysis (2 Marks)

1. New Input Problems

OCR errors:

OCR means Optical Character Recognition. It converts scanned images into editable text.

Common OCR errors include:

Correct text	**OCR output**	**Problem**

100	l00	Number 1 confused with letter l
500	5OO	Number 0 confused with letter O
Minimum	Minirnum	rn confused with m
Infection	Infectlon	Letter i confused with l
Patient	Pati-ent	Incorrect line-break hyphenation

Code-mixed language:

Example:

patient ku fever irundhuchu, paracetamol 500mg kuduthen

Meaning:

"The patient had fever, and I gave paracetamol 500 mg."

This sentence contains English medical terms mixed with Romanised Tamil words.

2. Original Assumptions That No Longer Work

Original assumption	**New problem**
Input is clean and typed	OCR introduces spelling errors
Vocabulary is mostly English	Regional-language words appear
Words are correctly separated	Words may be broken across lines
Exact matching is sufficient	Misspelt words require approximate matching
Lexicon is mostly closed	At least 20% of tokens may be unknown or non-English

3. Effect on System Performance

The new input is expected to reduce:

- Regular expression precision because incorrect characters may produce false matches.
- Recall because misspelt entities may not match existing patterns.
- Morphological coverage because unknown words may not exist in the lexicon.
- Retrieval accuracy because code-mixed words may not be understood.

The actual reduction must be calculated using the clean and noisy evaluation datasets.

B. Evaluation of the Original Design (2 Marks)

1. Three Example Records

Record 1 – OCR input

Patient ID: PT-l23456

Drug: Paracetamol 5OOmg

Diagnosis: Infectlon

Problems:

- l is mistaken for 1.
- OO is mistaken for 00.
- Infectlon contains an OCR character error.

The original exact-match regular expressions may fail to recognise the patient ID and drug strength.

Record 2 – Code-mixed input

patient ku fever irundhuchu

paracetamol 500mg kuduthen

Problems:

- ku, irundhuchu and kuduthen are not English words.
- The English morphological analyser cannot analyse regional-language forms.
- The system may recognise the drug but miss the clinical sentence meaning.

Record 3 – OCR and code-mixed input

Patient ku fever irundhuchu

Paracetamol 5OOmg kuduthen

Problems:

- The sentence contains mixed English and Tamil.
- The dosage contains an OCR error.
- The English morphological analyser cannot understand the complete sentence.

2. Performance Degradation Measurement

Compare the original clean corpus against a noisy test sample.

Evaluation measure	**Clean corpus**	**Noisy corpus**
Pattern precision	Actual result	Actual result

Pattern recall	Actual result	Actual result
Morphological coverage	Actual result	Actual result
Retrieval F1-score	Actual result	Actual result

These measurements should be obtained by testing the same patterns and analyser on both datasets.

3. Alternative Techniques

Technique	**Purpose**	**Benefit**
Weighted finite-state transducer	Handles alternative character sequences	Supports approximate matching
Edit distance	Measures spelling similarity	Corrects small spelling errors
Character n-gram language identification	Identifies language from character sequences	Helps detect code-mixed tokens
Expanded lexicon	Stores regional and medical words	Improves coverage
OCR correction	Corrects common character confusion	Improves structured entity extraction

C. Adaptation or Redesign (2 Marks)

1. Selected Approach: Modify the Original Design

I recommend modifying the original finite-state pipeline instead of completely replacing it.

The revised system will include OCR correction, approximate matching and language identification before morphological analysis.

2. Revised Architecture

Input sources

Typed English, OCR text and Romanised regional language

OCR error correction

Correct 0/O, 1/l, rn/m and broken words

Tokenisation and language identification

Separate English and regional-language tokens

Approximate matching

Weighted edit distance and spelling correction

Morphological analyser

Expanded English and regional-language lexicon

Normalised clinical records

Searchable stems, entities and language information

3. Changes in Each Component

Component	Original design	Modified design
Regular expressions	Exact matching	Exact matching plus tolerant candidate detection
Automata	Accept or reject	Weighted alternatives for uncertain matches
Lexicon	English clinical words	English, medical abbreviations and regional words
Morphological rules	English morphology	Extended language-specific morphology
Tokenisation	Standard word splitting	Handles OCR line breaks and hyphenation
Search	Exact and stem-based	Approximate and morphology-aware search

4. Expected Effects

Factor	Expected effect after modification
Coverage	Increases because unknown and regional words are handled better
Precision	May decrease if approximate matching accepts incorrect words
Runtime	Increases because additional correction stages are required
Maintenance	Increases because regional dictionaries and correction rules need updates
Annotation effort	Increases because noisy and code-mixed examples must be labelled

The actual amount of improvement should be reported after evaluating the revised system on the noisy sample.

D. Evidence-Based Defence (2 Marks)

1. Reasons for Choosing Modification

The modified design is suitable because:

1. Regular expressions are still effective for structured clinical entities.
2. Finite-state morphology efficiently handles known English word forms.
3. OCR correction helps recover incorrectly recognised characters.
4. Edit-distance matching identifies words with small spelling differences.
5. Language identification helps separate English and regional-language tokens.
6. An expanded lexicon improves morphological coverage.

2. Remaining Limitations

Even after modification, the system has limitations:

- Severe OCR errors may not be corrected accurately.
- Unknown regional-language words may remain unrecognised.
- The same word may have different meanings in different contexts.
- Approximate matching can introduce false positives.
- Finite-state systems cannot fully understand complex clinical meaning, intention or context on their own.

3. Final Recommendation

The hospital should deploy a hybrid finite-state normalisation system with OCR correction, weighted matching and language identification.

This approach retains the strengths of regular expressions and finite-state morphology while improving its ability to process noisy and code-mixed clinical records.

Conclusion

The new input requirements expose the limitations of the original English-only system. OCR errors and Romanised regional-language words reduce pattern matching accuracy and morphological coverage. Modifying the original pipeline with OCR correction, approximate matching, language identification and an expanded lexicon provides a practical solution. However, human review is still necessary for uncertain clinical information.

Pasted markdown(8).md

Final Assignment Completion Checklist

Task	Topic	Main requirement
1	Problem framing	Linguistic levels and pipeline

2	Regular expressions	12 clinical entity patterns
3	Tokenisation and normalisation	Tokenizer and normalisation rules
4	Finite-state automata	DFA, NFA, subset construction
5	Inflectional morphology	Paradigms and spelling rules
6	Derivational morphology	Affixes and word families
7	Finite-state morphological parsing	FST parsing and generation
8	Two-level morphology	Orthographic rules and transducers
9	Porter Stemmer	Stemming algorithm and evaluation
10	Implementation	Python code and testing
11	Comparison	Recommended normalisation design
12	Design challenge	OCR and code-mixed input adaptation